PRÁCTICA 3: EXPLORANDO UNA RED DE CANCIONES. TAD COLA Y TAD PILA

CONTEXTO DEL PROBLEMA

En esta práctica continuaréis desarrollando vuestro **reproductor de música por consola** y añadiréis las funcionalidades de **cola de reproducción** y **personalización de listas de favoritos**.

Además, seguiréis haciendo uso de las recomendaciones de canciones, que pueden verse como una red de canciones conectadas entre sí, para **mejorar la exploración musical** de una canción a otra. En total, los objetivos de esta práctica son:

1. Implementación y simulación de la cola de reproducción.
2. Exploración musical mejorada.
3. Creación y personalización de listas de favoritos.

Para implementar estos puntos, usaremos el **TAD Cola y el TAD Lista.**

OBJETIVOS

- Aprender un diseño e implementación del **TAD Cola (Queue)**.
- Comprender y aplicar el comportamiento **FIFO**.
- Comprender el uso y diferencias de los algoritmos de exploración (DFS y BFS).
- Aprender a diseñar e implementar el **TAD Lista (List).**
- Integrar las nuevas estructuras de datos con los TAD Music y Radio ya implementados.

NORMAS

Los programas que se entreguen deben seguir los criterios indicados en el Moodle de prácticas, considerando, además:

- Estar escritos en lenguaje C.
- Ejecutarse correctamente en una consola de comandos.
- Incorporar un adecuado control de errores (pensar tanto en casos base como extremos).
- Gestionar correctamente la memoria dinámica (**no debe haber fugas de memoria**).
- Respetar la **organización y nomenclatura de ficheros y prototipos** indicados en este enunciado.

PLANIFICACIÓN

- **Semana 1:** Ejercicio 1. Cola de reproducción.

- **Semana 2:** Ejercicio 2. Exploración musical mejorada (BFS).
- **Semana 3:** Ejercicio 3. Lista de favoritos.
- **Opcional:** Ejercicio 4. Implementación del TAD QueueList (cola usando lista).

La entrega final se realizará a través de Moodle, siendo la fecha límite en evaluación continua el **16 de abril** hasta la hora de inicio de la clase de prácticas. El fichero comprimido deberá seguir el formato `Px_apellido1_nombre1_apellido2_nombre2`, siendo `x` el número de pareja. Este fichero deberá contener, al menos, la siguiente organización de ficheros:

```
--- P1_apellido1_nombre1_.../
    |--- music.c
    |--- music.h
    |--- radio.c
    |--- radio.h
    |--- queue.h
    |--- queue.c
    |--- stack.h
    |--- libstack.a
    |--- list.h
    |--- list.c
    |--- types.h
    |--- p3_e1.c
    |--- p3_e2.c
    |--- p3_e3.c
    |--- Makefile
    |--- radio.txt
    |--- radio_bfs.txt
```

EJERCICIO 1. COLA DE REPRODUCCIÓN

DEFINICIÓN E IMPLEMENTACIÓN DE INTERFAZ DEL TAD QUEUE

En este primer ejercicio se va a realizar una implementación del **TAD Queue**, usando una variante de diseño donde el <u>front y el rear son de tipo puntero</u>. Para implementar el TAD `Queue` deberás:

- Definir en el fichero `queue.c` la estructura de datos `_Queue`:

```c
struct _Queue {
    void *data[MAX_QUEUE];
    void **front;
    void **rear;
};
```

Antes de programar, procura entender cómo funciona el diseño de cola propuesto

- Implementar en el fichero `queue.c` las funciones de la interfaz pública declaradas en el fichero `queue.h`.
- Incluir en el fichero `queue.c` las funciones privadas que consideres oportunas.

PRUEBA DE FUNCIONAMIENTO BÁSICO

Crea un programa `p3_e1.c` que reciba como argumento de entrada un fichero de radio y que realice las siguientes operaciones:

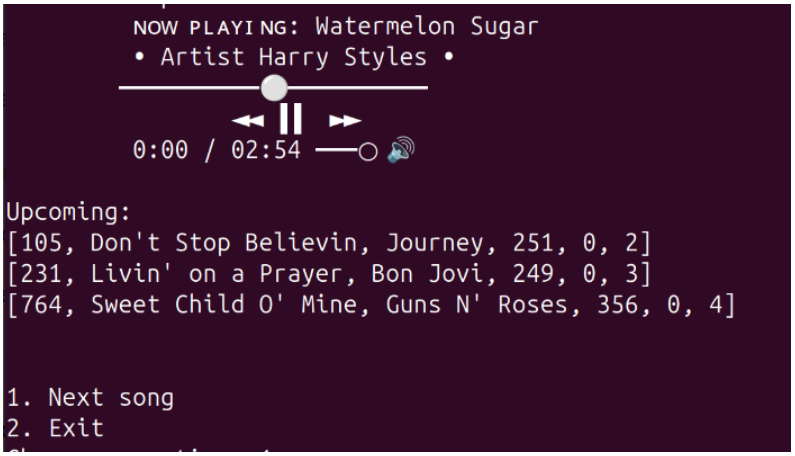
- Leer la radio desde fichero.
- Insertar las canciones de la radio en una cola de reproducción.
- Ejecutar una rutina de menú interactivo (`now_playing_menu`) y realizar las acciones correspondientes según la opción seleccionada:
    - 'Next' *reproduce* (imprime) la siguiente canción (esto es, pop de cola y nueva impresión del menú)
    - 'Exit' libera recursos y finaliza la ejecución

La rutina `now_playing_menu` debe implementarse de forma privada en el main y ser similar a `show_player_menu` de la práctica anterior. Su prototipo será:

```
int now_playing_menu(Queue *q);
```

Salida esperada:

Y tras seleccionar '1':



EJERCICIO 2. EXPLORACIÓN MUSICAL MEJORADA

BREADTH-FIRST SEARCH (BFS)

La exploración sobre redes es algo común en sistemas reales, ya sea desde motores de recomendación musical hasta redes sociales o buscadores web. En nuestro caso, las relaciones de cada canción con otras forman la red de canciones que nos permite explorar caminos musicales.

En la práctica anterior usamos **depth-first search (DFS)** para recorrer esta red de canciones usando una **pila**. Sin embargo, el camino de exploración de una canción a otra puede no ser el mejor con este algoritmo. Un ejemplo puede verse en el siguiente caso:

Dadas las siguientes relaciones entre canciones y la búsqueda del camino de **A** a **G**

A → F → G

A → B → C → D → E

Si no se exploraran las relaciones en orden alfabético, DFS insertaría F y luego B en la pila, resultando en la exploración profunda de la segunda rama (seguiría buscando por la rama de B todo lo lejos que pudiera llegar antes de volver y probar otro camino). Por tanto, tardaría mucho en llegar de A a G.

Para mejorar la exploración musical, usaremos en esta práctica el algoritmo **breadth-first search (BFS)** que explora la red por niveles y no por profundidad. En el caso de las relaciones de canciones anterior, la exploración sería A **(nivel 0)**, luego F y B **(nivel 1)**, y terminaría con G y C **(nivel 2)**:

A → F → G

A → B → C → D → E

De esta forma, BFS nos da el camino más corto y no un camino cualquiera como DFS.

Codifica el algoritmo de BFS, con el respectivo control de errores, mediante la función:

```
Status radio_breadthSearch (Radio *r, long from_id, long to_id);
```

Para implementar esta rutina, parte del código del algoritmo de DFS y reemplaza el uso de la pila por una cola. Este sencillo cambio nos permite entender cómo la **estructura de datos cambia completamente el comportamiento del algoritmo**.

Por último, **responde** a las siguientes cuestiones al final de la documentación de cabecera de la rutina `radio_breadthSearch`:

1. ¿Cuál es la principal diferencia de comportamiento entre DFS y BFS?
2. ¿En qué tipos de problemas crees que funcionaría mejor un algoritmo frente al otro? Indica, al menos, 2 ejemplos por cada uno.

Si estás leyendo esto, no olvides responder a las preguntas porque puntúan

PRUEBA DE FUNCIONAMIENTO BÁSICO

Crea un programa `p3_e2.c` que utilice la función `radio_ breadthSearch` del TAD Radio para recorrer una radio contenida en el fichero pasado como argumento, desde la canción origen (id pasado como argumento) a la canción destino (id pasado como argumento).

El programa deberá:

- Imprimir la radio de canciones
- Imprimir el resultado de aplicar BFS desde el id origen hasta el id destino
- Imprimir el resultado de aplicar DFS desde el id origen hasta el id destino

Salida esperada:

```
Radio:
[1, Bohemian Rhapsody, Queen, 354, 0, 0]: [2, Hotel California, Eagles, 391, 0, 1] [3, Imagine, Joh
n Lennon, 183, 0, 2]
[2, Hotel California, Eagles, 391, 0, 1]: [4, Like a Rolling Stone, Bob Dylan, 369, 0, 3] [5, Smell
s Like Teen Spirit, Nirvana, 301, 0, 4]
[3, Imagine, John Lennon, 183, 0, 2]: [6, Billie Jean, Michael Jackson, 294, 0, 5]
[4, Like a Rolling Stone, Bob Dylan, 369, 0, 3]: [7, Hey Jude, The Beatles, 431, 0, 6]
[5, Smells Like Teen Spirit, Nirvana, 301, 0, 4]: [8, Sweet Child O' Mine, Guns N' Roses, 356, 0, 7
]
[6, Billie Jean, Michael Jackson, 294, 0, 5]: [9, Wonderwall, Oasis, 258, 0, 8]
[7, Hey Jude, The Beatles, 431, 0, 6]:
[8, Sweet Child O' Mine, Guns N' Roses, 356, 0, 7]: [10, Stairway to Heaven, Led Zeppelin, 482, 0,
9]
[9, Wonderwall, Oasis, 258, 0, 8]: [10, Stairway to Heaven, Led Zeppelin, 482, 0, 9]
[10, Stairway to Heaven, Led Zeppelin, 482, 0, 9]:

--------DFS------------
From music id: 1
To music id: 2
Output:
[1, Bohemian Rhapsody, Queen, 354, 1, 0]
[3, Imagine, John Lennon, 183, 1, 2]
[6, Billie Jean, Michael Jackson, 294, 1, 5]
[9, Wonderwall, Oasis, 258, 1, 8]
[10, Stairway to Heaven, Led Zeppelin, 482, 1, 9]
[2, Hotel California, Eagles, 391, 1, 1]

--------BFS------------
From music id: 1
To music id: 2
Output:
[1, Bohemian Rhapsody, Queen, 354, 1, 0]
[2, Hotel California, Eagles, 391, 1, 1]
```

EJERCICIO 3. LISTA DE FAVORITOS

DEFINICIÓN E IMPLEMENTACIÓN DE INTERFAZ DEL TAD LIST

En este ejercicio se va a realizar una implementación del **TAD List**, definida como una **lista enlazada simple con acceso frontal y posterior**. Por tanto, la implementación del `TAD List` debe contar con un puntero al primer nodo de la lista (front) y otro al último nodo de la lista (rear).

Además, cada nodo de la lista será una estructura capaz de almacenar un dato genérico y un puntero al siguiente elemento de la lista.

Los pasos que considerar en la implementación son:

- Definir en el fichero `list.c` la estructura de datos `_List` y `_Node`:

```
struct _Node {
    void *info;
    struct _Node *next;
};

typedef struct _Node Node;

struct _List {
    Node *first; /* First node of the list. */
    Node *last; /* Last node of the list. */
    int size;
};
```

Antes de programar, procura entender cómo funciona el diseño de lista propuesto

- Implementar en el fichero `list.c` las funciones de la interfaz pública declaradas en el fichero `list.h`.
- Incluir en el fichero `list.c` las funciones privadas que consideres oportunas.

Al igual que en implementaciones de TAD previos (e.g. stack, queue), al insertar un elemento en una lista no se reserva nueva memoria para el dato ni se hace una copia, simplemente se la referencia (el puntero) al elemento a insertar.

PRUEBA DE FUNCIONAMIENTO BÁSICO

Para comprobar que se ha implementado correctamente el TAD, se realizará un programa `p3_e3` que recibirá un fichero de radio como argumento de entrada (por ejemplo, radio_bfs.txt). Se leerán las canciones del fichero de manera que la primera, tercera, quinta, etc. se insertarán por el final de una lista, mientras que las demás (segunda, cuarta, sexta, etc.) se insertarán por el principio. Cuando se haya terminado de insertar, el programa irá extrayendo los elementos de la lista de uno en uno: la primera mitad desde el principio y la segunda mitad por el final.

Salida esperada:

```
[10, Stairway to Heaven, Led Zeppelin, 482, 0, 0]
[8, Sweet Child O' Mine, Guns N' Roses, 356, 0, 0]
[6, Billie Jean, Michael Jackson, 294, 0, 0]
[4, Like a Rolling Stone, Bob Dylan, 369, 0, 0]
[2, Hotel California, Eagles, 391, 0, 0]
[1, Bohemian Rhapsody, Queen, 354, 0, 0]
[3, Imagine, John Lennon, 183, 0, 0]
[5, Smells Like Teen Spirit, Nirvana, 301, 0, 0]
[7, Hey Jude, The Beatles, 431, 0, 0]
[9, Wonderwall, Oasis, 258, 0, 0]

Finished inserting. Now we extract from the beginning:
[10, Stairway to Heaven, Led Zeppelin, 482, 0, 0] [8, Sweet Child O' Mine, Guns N' Roses,
356, 0, 0] [6, Billie Jean, Michael Jackson, 294, 0, 0] [4, Like a Rolling Stone, Bob Dyla
n, 369, 0, 0] [2, Hotel California, Eagles, 391, 0, 0]

Now we extract from the end:
[9, Wonderwall, Oasis, 258, 0, 0] [7, Hey Jude, The Beatles, 431, 0, 0] [5, Smells Like Te
en Spirit, Nirvana, 301, 0, 0] [3, Imagine, John Lennon, 183, 0, 0] [1, Bohemian Rhapsody,
 Queen, 354, 0, 0]
```

EJERCICIO 4 (OPCIONAL)

En este ejercicio se desea implementar un nuevo TAD Cola a partir del TAD Lista. Para ello, se define en un fichero `queueList.c` el nuevo tipo de dato como:

```
struct _Queue {
   List *pl;
};
```

Al igual que hemos hecho anteriormente, se debe implementar en `queueList.c` la interfaz publica de `queue.h,` así como las funciones privadas que se consideres necesarias.

Para probar el correcto funcionamiento del nuevo TAD, basta con añadir objetivos al Makefile que creen ejecutables los ejercicios p3_e1s y p3_e2s, que se distinguen de sus versiones porque enlazarán sus programas con la nueva implementación de la cola.

En este nuevo TAD estamos haciendo un encapsulamiento del TAD List dentro del TAD cola, aprovechando las rutinas de push_rear y pop_front que nos permiten simular el comportamiento normal de una cola.